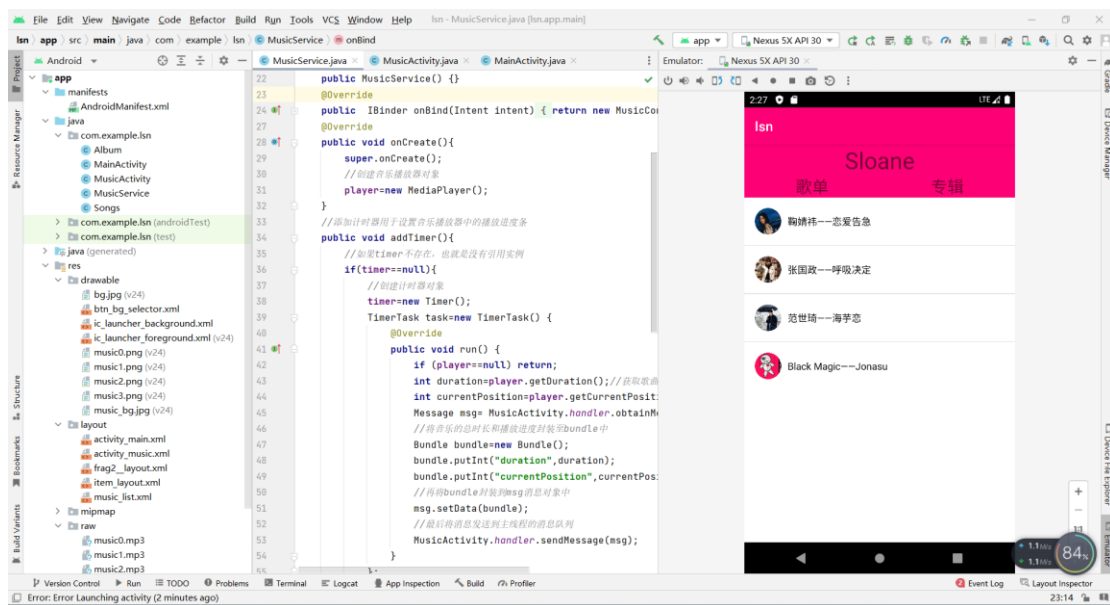
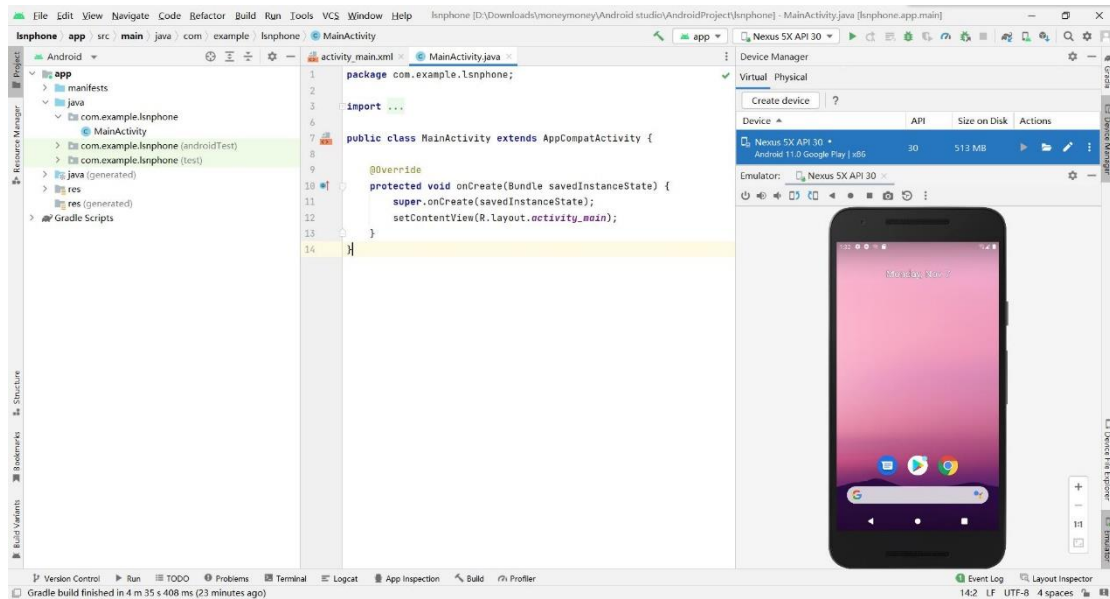


## 第一章作业：

P22 1. 下载最新版本的 Android Studio，安装后如果需要更新 Gradle，则将其更新到最新版本。

P22 2. 创建一个任意类型的手机 App 工程，分别使用虚拟设备和物理设备运行。



## 第二章作业:

P73 2.2 实现多选题的界面效果。

代码:

```
public class MainActivity extends AppCompatActivity implements
CompoundButton.OnCheckedChangeListener {
    private final ArrayList<String> mHobbies = new ArrayList<>();
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        final TextView resultTextView =
findViewById(R.id.textView);
        CheckBox checkBox1 = findViewById(R.id.checkBox5);
        CheckBox checkBox2 = findViewById(R.id.checkBox4);
        CheckBox checkBox3 = findViewById(R.id.checkBox3);
        CheckBox checkBox4 = findViewById(R.id.checkBox6);
        Button submitButton = findViewById(R.id.button);
        checkBox1.setOnCheckedChangeListener(this); //CheckBox 设置监
听器
        checkBox2.setOnCheckedChangeListener(this);
        checkBox3.setOnCheckedChangeListener(this);
        checkBox4.setOnCheckedChangeListener(this);
        submitButton.setOnClickListener(v -> { //Button 设置监听器
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < mHobbies.size(); i++) {
                if (i == (mHobbies.size() - 1)) {
                    sb.append(mHobbies.get(i));
                } else {
                    sb.append(mHobbies.get(i)).append(", ");
                }
            }
            if (sb.length() == 0) { //显示选择结果
                resultTextView.setText("您还没有进行选择了!");
            } else {
                resultTextView.setText("您选择了: " + sb + "。");
            }
        });
        public void onCheckedChanged(CompoundButton compoundButton,
boolean isChecked) {
            if (isChecked) {
                mHobbies.add(compoundButton.getText().toString().trim());
            } else {
                mHobbies.remove(compoundButton.getText().toString().trim());
            }
        }
}

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
```

```

xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">
<CheckBox
    android:id="@+id/checkBox5"
    android:layout_width="352dp"
    android:layout_height="78dp"
    android:rotationX="-1"
    android:textSize="30dp"
    android:text="钓鱼"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.493"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.104" />
<CheckBox
    android:id="@+id/checkBox4"
    android:layout_width="352dp"
    android:layout_height="78dp"
    android:rotationX="-1"
    android:text="游泳"
    android:textSize="30dp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.491"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.269" />
<CheckBox
    android:id="@+id/checkBox3"
    android:layout_width="352dp"
    android:layout_height="78dp"
    android:rotationX="-1"
    android:textSize="30dp"
    android:text="喝水"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.491"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"

```

```

        app:layout_constraintVertical_bias="0.415" />
<CheckBox
    android:id="@+id/checkBox6"
    android:layout_width="352dp"
    android:layout_height="78dp"
    android:rotationX="-1"
    android:textSize="30dp"
    android:text="篮球"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.491"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.572" />
<TextView
    android:id="@+id/textView"
    android:layout_width="362dp"
    android:layout_height="43dp"
    android:gravity="center"
    android:text="你的爱好是什么?"
    android:textSize="20dp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.023" />
<Button
    android:id="@+id/button"
    android:layout_width="164dp"
    android:layout_height="66dp"
    android:layout_marginTop="48dp"
    android:layout_marginBottom="183dp"
    android:text="确定"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.497"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/checkBox6"
    app:layout_constraintVertical_bias="0.0" />
</androidx.constraintlayout.widget.ConstraintLayout>

```

运行结果：



你的爱好是什么？

☒ 钓鱼

☒ 游泳

☐ 喝水

☐ 篮球



p73 2.3 实现选择出生日期的界面效果。

代码:

```
public class MainActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        final DatePicker datePicker =
findViewById(R.id.date_picker);
        datePicker.updateDate(2021, 0, 1);
        datePicker.setOnDateChangeListener((view, year,
monthOfYear, dayOfMonth) -> {
            Calendar calendar = Calendar.getInstance();
            calendar.set(year, monthOfYear, dayOfMonth);
            @SuppressWarnings("SimpleDateFormat") SimpleDateFormat
simpleDateFormat = new SimpleDateFormat("yyyy年MM月dd日");
            Toast.makeText(getApplicationContext(), "当前选择时间是: "
+ simpleDateFormat.format(calendar.getTime()),
Toast.LENGTH_SHORT).show();
        });
        Button button1 = findViewById(R.id.button1);
        button1.setOnClickListener(v -> {
            datePicker.updateDate(2020, 0, 1);
            Toast.makeText(getApplicationContext(), "已经恢复到默认时
间!", Toast.LENGTH_LONG).show();
        });
        Button button2 = findViewById(R.id.button2);
        button2.setOnClickListener(v -> {
            Calendar calendar = Calendar.getInstance();
            calendar.set(datePicker.getYear(),
datePicker.getMonth(), datePicker.getDayOfMonth());
            @SuppressWarnings("SimpleDateFormat") SimpleDateFormat
simpleDateFormat = new SimpleDateFormat("yyyy年MM月dd日");
            Toast.makeText(getApplicationContext(), "你的出生日期是: "
+ simpleDateFormat.format(calendar.getTime()),
Toast.LENGTH_SHORT).show();
        }); }
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">
<TextView
```

```
        android:id="@+id/textView3"
        android:layout_width="425dp"
        android:layout_height="41dp"
        android:gravity="center"
        android:text="请选择你的出生日期: " />
    <DatePicker
        android:id="@+id/date_picker"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:calendarViewShown="false"
        android:datePickerMode="spinner"
        android:startYear="2020" />
    <Button
        android:id="@+id/button1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="重置" />
    <Button
        android:id="@+id/button2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="确定" />
</LinearLayout>
```

运行结果：

12:56

LTE

chushengriqi

请选择你的出生日期：

|     |    |      |
|-----|----|------|
| Jan | 29 |      |
| Feb | 01 | 2020 |
| Mar | 02 | 2021 |

重置

确定

当前选择时间是：2020年02月01日



### 第三章

#### 3.1 实现注册界面的布局效果，至少包括用户名、密码、忘记密码、登录等控件。

代码：

```
public class MainActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main); }}

<?xml version="1.0" encoding="utf-8"?>
<TableLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="300dp"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:collapseColumns="2"
    android:stretchColumns="1">
    <TableRow><TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="用户名" />
    <EditText
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:ems="10"
        android:inputType="textPersonName" />
</TableRow> <TableRow>
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="密码" />
    <EditText
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:ems="10"
        android:inputType="textPassword" />
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="忘记密码" />
</TableRow> <TableRow>
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="忘记密码" />
```

```

<TextView
    android:id="@+id/textView4"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_column="1"
    android:gravity="end"
    android:text="注册" />
</TableRow>
<TableRow android:orientation="vertical">
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_span="2"
        android:text="登录" />
</TableRow></TableLayout>

```

运行结果：



用户名

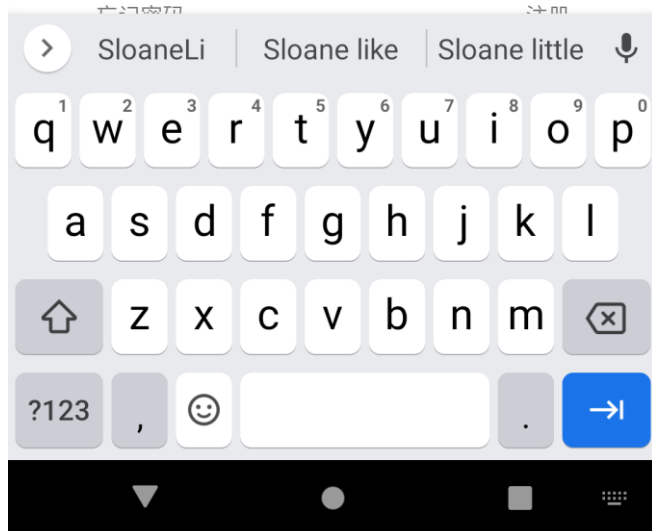
密码

忘记密码 [注册](#)



用户名 SloaneLi

密码



#### 第四章:

p122 4.3 实现京东首页的轮播广告效果, 至少包括 3 个产品广告的图片。

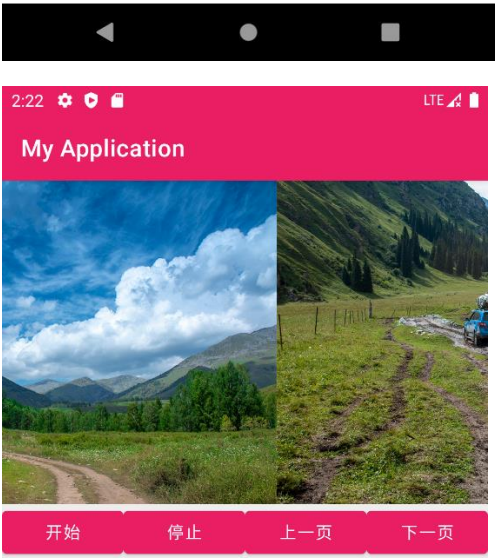
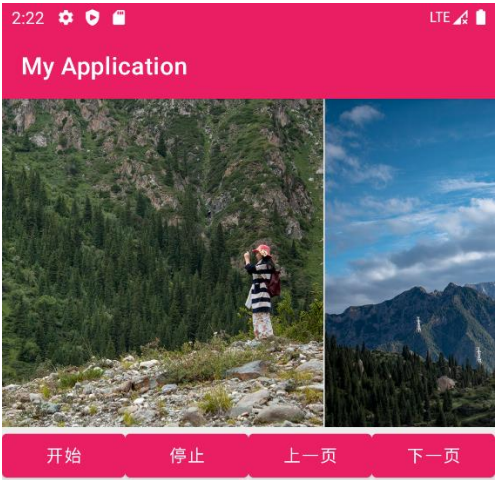
代码:

```
public class MainActivity extends AppCompatActivity {
    private ViewFlipper mViewFlipper;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        View view3 = View.inflate(MainActivity.this,
R.layout.view_flipper_page3, null);
        ImageView imageView4 = new ImageView(this);
        imageView4.setAdjustViewBounds(true);
        imageView4.setImageResource(R.mipmap.img_9546);
        mViewFlipper = findViewById(R.id.view_flipper);
        mViewFlipper.addView(imageView4);
        mViewFlipper.addView(view3, 2);
        mViewFlipper.startFlipping();
        mViewFlipper.setOnClickListener(v ->
Toast.makeText(MainActivity.this, "当前显示的子视图索引号为
"+mViewFlipper.getDisplayedChild(), Toast.LENGTH_SHORT).show());
        Button startBtn = findViewById(R.id.button_start); //开始按钮
        startBtn.setOnClickListener(v ->
mViewFlipper.startFlipping());
        Button stopBtn = findViewById(R.id.button_stop); //停止按钮
        stopBtn.setOnClickListener(v ->
mViewFlipper.stopFlipping());
        Button previousBtn = findViewById(R.id.button_previous);
        previousBtn.setOnClickListener(v -> //上一页按钮
mViewFlipper.showPrevious());
        Button nextBtn = findViewById(R.id.button_next); //下一页按钮
        nextBtn.setOnClickListener(v -> mViewFlipper.showNext());
    }
}

<?xml version="1.0" encoding="utf-8"?>
<GridLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="top|fill"
    android:alignmentMode="alignMargins"
    android:background="#eeeeee"
    android:columnCount="4"
```

```
        android:rowCount="2">
    <ViewFlipper
        android:id="@+id/view_flipper"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_columnSpan="4"
        android:flipInterval="3000"
        android:inAnimation="@anim/view_flipper_right_in"
        android:outAnimation="@anim/view_flipper_left_out">
        <include layout="@layout/view_flipper_page1" />
        <include layout="@layout/view_flipper_page2" />
    </ViewFlipper>
    <Button
        android:id="@+id/button_start"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_columnWeight="1.0"
        android:text="开始" />
    <Button
        android:id="@+id/button_stop"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_columnWeight="1.0"
        android:text="停止" />
    <Button
        android:id="@+id/button_previous"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_columnWeight="1.0"
        android:text="上一页" />
    <Button
        android:id="@+id/button_next"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_columnWeight="1.0"
        android:text="下一页" />
</GridLayout>
```

运行结果：



## 第五章:

p150 5.1 实现登录后显示用户名的效果。输入用户名、密码后登录，启动一个新的Activity显示用户名。

代码:

```
public class MainActivity extends AppCompatActivity {
    private static final int REQUEST_CODE = 101;
    private EditText mResultEditText;
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        mResultEditText = findViewById(R.id.edit_text_result);
        findViewById(R.id.button_send).setOnClickListener(new
View.OnClickListener() { //发送数据（没有回传数据）
            public void onClick(View v) {
                Intent intent = new Intent(MainActivity.this,
ReceiveActivity.class);
                intent.putExtra("id", 10407);
                intent.putExtra("msg", "MainActivity 发送的信息 1");
                startActivity(intent);
                mResultEditText.setText("");});});
        findViewById(R.id.button_send_for_result).setOnClickListener(new
View.OnClickListener() {
            public void onClick(View view) {
                Intent intent = new Intent(MainActivity.this,
ReceiveActivity.class);
                intent.putExtra("id", 20408);
                intent.putExtra("msg", "MainActivity 发送的信息 2");
                startActivityForResult(intent, REQUEST_CODE);
                mResultEditText.setText("");});}); }
    protected void onActivityResult(int requestCode, int
resultCode, Intent data) { //处理回调数据
        super.onActivityResult(requestCode, resultCode, data);
        mResultEditText.setText("正在处理回调数据");
        if (requestCode == REQUEST_CODE) { //判断请求码
            if (resultCode == ReceiveActivity.RESULT_CODE) {
                String result = data.getStringExtra("data");
                mResultEditText.setText(result);
            }else{
                mResultEditText.setText("没有回传数据");}}}}
    public class ReceiveActivity extends AppCompatActivity {
        public static final int RESULT_CODE = 201;
        private EditText mIdEditText;
        private EditText mMsgEditText;
        protected void onCreate(Bundle savedInstanceState) {
```

```

super.onCreate(savedInstanceState);
setContentView(R.layout.activity_receive);
Intent intent = getIntent();//获取传递数据
int id = intent.getIntExtra("id", 0);
String msg = intent.getStringExtra("msg");
mIdEditText = findViewById(R.id.edit_text_id); //显示传递数据
mIdEditText.setText(String.valueOf(id));
mMsgEditText = findViewById(R.id.edit_text_name);
mMsgEditText.setText(msg);
findViewById(R.id.button_finish).setOnClickListener(view ->
{
    finish();//关闭当前 Activity});
findViewById(R.id.button_finish_result).setOnClickListener(view
-> {
    Intent intent1 = new Intent();//设置返回的数据//关闭（发送回传数
据）

    intent1.putExtra("data", "已经查阅信息！");
    setResult(RESULT_CODE, intent1);
    finish();//关闭当前 Activity}}});

```

运行结果：







## 第六章:

### p176 6.3 通过广播接收器实现两个 App 之间通过自定义广播进行数据通信。

代码:

```
public class MainActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        findViewById(R.id.button_send_broadcast).setOnClickListener(new
        View.OnClickListener() { //发送标准广播
            public void onClick(View v) {
                Intent intent=new Intent();
                intent.setAction("MyBroadcastReceiver.Custom");
                intent.putExtra("info","悄悄地告诉你一个秘密^6^");

                intent.addFlags(Intent.FLAG_ACTIVITY_PREVIOUS_IS_TOP);
                sendBroadcast(intent); } });
        findViewById(R.id.button_send_ordered_broadcast).setOnClickListener(new
        View.OnClickListener() { //发送有序广播（连续传递）
            public void onClick(View v) {
                Intent intent=new Intent();
                intent.setAction("OrderedBroadcast.Custom");
                intent.putExtra("info","悄悄地告诉你一个秘密^6^");

                intent.addFlags(Intent.FLAG_ACTIVITY_PREVIOUS_IS_TOP);
                sendOrderedBroadcast(intent,null); } });
        findViewById(R.id.button_send_ordered_broadcast_stop).setOnClickListener(new
        View.OnClickListener() { //发送有序广播（传递一次）
            public void onClick(View v) {
                Intent intent=new Intent();
                intent.setAction("OrderedBroadcast.Custom");
                intent.putExtra("stop",true);
                intent.putExtra("info","悄悄地告诉你一个秘密 不要告诉别人哦
                ^6^");
                intent.addFlags(Intent.FLAG_ACTIVITY_PREVIOUS_IS_TOP);
                sendOrderedBroadcast(intent,null); } });
    }
}

public class MyBroadcastReceivers1 extends BroadcastReceiver {
    private static final String TAG = "MyBroadcastReceiver";
    public void onReceive(Context context, Intent intent) {
        Log.d(TAG, "-----Receivers1-----");
        Log.d(TAG, "Action: " + intent.getAction());
        Log.d(TAG, "URI: " +
        intent.toUri(Intent.URI_INTENT_SCHEME));
        Log.d(TAG, "自定义广播的 info: " +
        intent.getStringExtra("info")); } }
```

```
public class MyBroadcastReceivers2 extends BroadcastReceiver {
    private static final String TAG = "MyBroadcastReceiver";
    public void onReceive(Context context, Intent intent) {
        Log.d(TAG, "-----Receivers2-----");
        Log.d(TAG, "Action: " + intent.getAction());
        Log.d(TAG, "URI: " +
intent.toUri(Intent.URI_INTENT_SCHEME));
        Log.d(TAG, "自定义广播的 info: "
+intent.getStringExtra("info"));
        if(intent.getBooleanExtra("stop", false)){
            abortBroadcast();}}}

```

运行结果:



C0606

发送标准广播

发送有序广播（连续传递）

发送有序广播（传递一次）

## 第七章:

p213 7.2 使用 SQLite 实现记录登录时间的功能, 并且能够查询登录的历史记录。

代码:

```
public class SQLiteDBHelper extends SQLiteOpenHelper {
    static final String CREATE_SQL[] = { // 创建数据库
        "CREATE TABLE news (" +
        "_id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT," + "title"
        "varchar," + "content varchar," + "keyword varchar," + "category"
        "varchar," + "author INTEGER," + "publish_time varchar" + ")",
        "CREATE TABLE user (" + "_id INTEGER NOT NULL PRIMARY"
        "KEY AUTOINCREMENT," + "name varchar," + "password varchar,"
        + "email varchar," + "phone varchar," + "address varchar" + ")",
        "INSERT INTO user VALUES (1, 'admin', 123, 'admin@163.com', 123, '沈阳'
        ')",
        "INSERT INTO user VALUES
        (2, 'zhangsan', 123, 'zhangsan@163.com', 123, '北京')",
        "INSERT INTO user VALUES (3, 'lisi', 123, 'lisi@163.com', 123, '上海'
        ')",
        "INSERT INTO user VALUES (4, 'wangwu', 123, 'wangwu@163.com', 123, '深圳'
        ')" };

    public SQLiteDBHelper(@Nullable Context context, @Nullable
    String name, int version) { // 调用父类的构造方法 (便于之后进行初始化赋值)
        super(context, name, null, version); }

    public void onCreate(SQLiteDatabase db) {
        Log.i("sqlite_____", "create Database"); // 创建数据库
        for (int i = 0; i < CREATE_SQL.length; i++) { // 执行 SQL 语句
            db.execSQL(CREATE_SQL[i]); }
        Log.i("sqlite_____", "Finished Database"); // 完成数据库的创建
    }

    public class SQLiteListActivity extends AppCompatActivity {
        ListView listView01;
        SQLiteDBHelper dbHelper;

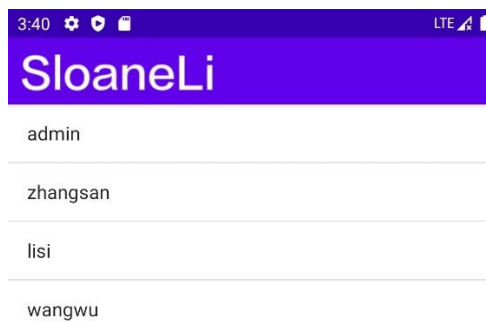
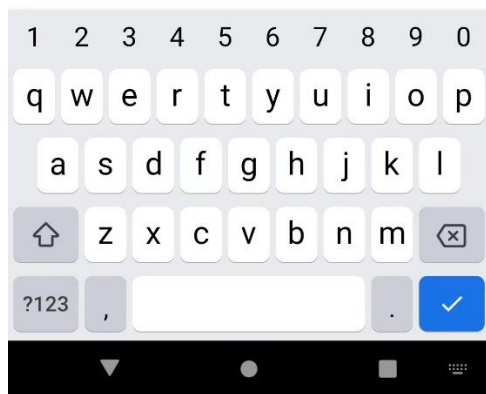
        protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_sqlite_list);
            Toast.makeText(this, "登录成功!", Toast.LENGTH_LONG).show();
            listView01 = findViewById(R.id.listView01);
            dbHelper = new SQLiteDBHelper(this, "sqlite.db", 1);
            SQLiteDatabase db = dbHelper.getReadableDatabase();
            Cursor cursor = db.query("user", new
            String[] { "_id", "name" }, null, null, null, null, null);
            SimpleCursorAdapter adapter = new SimpleCursorAdapter(this, andr
            oid.R.layout.simple_list_item_1, cursor, new
            String[] { "_id", "name" }, new
            int[] { android.R.id.text1, android.R.id.text1 }, 0);
```

```

        listView01.setAdapter(adapter);}
protected void onDestroy() {
    super.onDestroy();
    dbHelper.close();}}
public class SQLiteLoginActivity extends AppCompatActivity {
    SQLiteDBHelper dbHelper;
    EditText etUsername;
    EditText etPassword;
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_sqlite_login);
    dbHelper = new SQLiteDBHelper(this,"sqlite.db",1);
    etUsername = findViewById(R.id.etUsername);
    etPassword = findViewById(R.id.etPassword);}
public void onClick(View view) {
    String username = etUsername.getText().toString();
    String password = etPassword.getText().toString();
    SQLiteDatabase db = dbHelper.getReadableDatabase();
    Cursor cursor = db.query("user",new
String[]{"name","password"}, "name=?",new
String[]{username},null,null,null,"0,1");
if(cursor.moveToNext()) {@SuppressWarnings("Range") String dbPassword
= cursor.getString(cursor.getColumnIndex("password"));
    cursor.close();
    if(password.equals(dbPassword)) {
        Intent intent = new Intent(this,
SQLiteListActivity.class);
        startActivity(intent); // 启动
        return;}}
    cursor.close();// 跳转失败也要进行关闭
    Toast.makeText(this,"输入信息有误!
",Toast.LENGTH_LONG).show();}
public void onDestroy() {
    super.onDestroy();
    dbHelper.close();}}

```

运行结果:



## 第八章:

### p271 8.1 使用 Camera2 实现定时拍照的功能。

代码:

```
public void onClick(View view) {
    takePhoto();
}

@Override
@SuppressLint("MissingPermission") // 打开摄像头
private void openCamera(int width, int height) {
    Log.d(TAG, "openCamera()");
    if (!Permissions.hasPermissions(mContext,
PERMISSIONS)) {Permissions.requestPermissions(mContext,
PERMISSIONS);
        return;}
    try { // 2500 毫秒内请求获取 1 个许可, 否则抛出异常。
        if (!mSemaphore.tryAcquire(2500,
TimeUnit.MILLISECONDS)) {
            throw new RuntimeException("打开摄像头超时");}
        mCameraManager = (CameraManager)
getSystemService(Context.CAMERA_SERVICE);
        CameraCharacteristics characteristics = // 获取摄像头支持的属性
特征
mCameraManager.getCameraCharacteristics(mCameraId);
        StreamConfigurationMap map =
characteristics.get(CameraCharacteristics.SCALER_STREAM_CONFIGURA
TION_MAP); // 获取摄像头捕捉的最大尺寸读取图片
        Size largest =
Collections.max(Arrays.asList(map.getOutputSizes(ImageFormat.JPEG
)), new Util.CompareSizesByArea());
        mImageReader =
ImageReader.newInstance(largest.getWidth(), largest.getHeight(),
ImageFormat.JPEG, 2); // 选择最佳预览尺寸
        mPreviewSize = Util.chooseOptimalSize(mContext,
map.getOutputSizes(SurfaceTexture.class), width, height,
MAX_PREVIEW_WIDTH, MAX_PREVIEW_HEIGHT, largest);
        mPreviewView.setAspectRatio(mPreviewSize.getHeight(),
mPreviewSize.getWidth()); // 设置预览尺寸
        mCameraManager.openCamera(mCameraId, //打开摄像头
mCameraDeviceStateCallback, mBackgroundHandler);
    } catch (CameraAccessException e) {
        e.printStackTrace();
    } catch (InterruptedException e) {
        throw new RuntimeException("打开摄像头被中断", e);}
}

private void startPreview() { // 创建摄像头预览会话
    Log.d(TAG, "startPreview()");
}
```



```

        if (null == mCameraDevice || !mPreviewView.isAvailable() ||
null == mPreviewSize) { return; }
        try {
            closePreview();// 创建摄像头预览会话
            SurfaceTexture texture =
mPreviewView.getSurfaceTexture();
            texture.setDefaultBufferSize(mPreviewSize.getWidth(),
mPreviewSize.getHeight());// 设置预览输出的 Surface
            Surface surface = new Surface(texture);
            mCaptureRequestBuilder =
mCameraDevice.createCaptureRequest(CameraDevice.TEMPLATE_PREVIEW)
;

            mCaptureRequestBuilder.addTarget(surface);
mCameraDevice.createCaptureSession(Arrays.asList(surface,
mImageReader.getSurface()), new
CameraCaptureSession.StateCallback() {    public void
onConfigured(CameraCaptureSession cameraCaptureSession) {
                mCaptureSession = cameraCaptureSession;
                updatePreview();}
            public void onConfigureFailed(CameraCaptureSession
cameraCaptureSession) {
                Toast.makeText(mContext, "摄像头配置失败",
Toast.LENGTH_LONG).show();}}, mBackgroundHandler);
        } catch (CameraAccessException e) {
            e.printStackTrace();}
        private void takePhoto() {    // 拍照
            Log.d(TAG, "takePhoto()");
            if (null == mCameraDevice || !mPreviewView.isAvailable() ||
null == mPreviewSize) {
                return;}
            try {
                closePreview();
                mCaptureRequestBuilder =
mCameraDevice.createCaptureRequest(CameraDevice.TEMPLATE_STILL_CA
PTURE); // 设置预览的缓冲区大小
                SurfaceTexture texture =
mPreviewView.getSurfaceTexture();
                texture.setDefaultBufferSize(mPreviewSize.getWidth(),
mPreviewSize.getHeight());// 设置预览的 Surface
                List<Surface> surfaces = new ArrayList<>();
                Surface previewSurface = new Surface(texture);
                surfaces.add(previewSurface);
                mCaptureRequestBuilder.addTarget(previewSurface);
                Surface imageReaderSurface = mImageReader.getSurface();

```

```

        surfaces.add(imageReaderSurface);
        mCaptureRequestBuilder.addTarget(imageReaderSurface);
        mCameraDevice.createCaptureSession(surfaces, new
CameraCaptureSession.StateCallback() {
            public void onConfigured(@NonNull
CameraCaptureSession cameraCaptureSession) {
                Log.d(TAG,
"CameraCaptureSession.onConfigured()");
                mCaptureSession = cameraCaptureSession;
mImageReader.setOnImageAvailableListener(mOnImageAvailableListene
r, mBackgroundHandler);
                mCaptureRequest = mCaptureRequestBuilder.build();
                try { // 拍摄照片
                    mCaptureSession.capture(mCaptureRequest, null,
null);

                    } catch (CameraAccessException e) {
                        e.printStackTrace();}
                public void onConfigureFailed(CameraCaptureSession
cameraCaptureSession) {
                    Toast.makeText(mContext, "摄像头配置失败",
Toast.LENGTH_LONG).show();}, mBackgroundHandler);
                } catch (CameraAccessException e) {
                    e.printStackTrace();}
                public static void requestPermissions(final Context context,
String[] permissions) {
                    if (shouldShowRequestPermissionRationale(context,
permissions)) {
                        String msg = "";
                        for (int i = 0; i < permissions.length; i++) {
                            if (i == 0) {
                                msg = permissions[i];
                            } else {
                                msg = msg + "\n" + permissions[i];}
                        }
                    }
                }
            }
        }
    }
}

```

运行结果：

